

POLITECNICO DI TORINO



**Politecnico
di Torino**

Lab 3

Music and multimedia streaming over the Internet

Academic year 2025/2026

Full Name : MEHRAN BAYAT

Student Number : S320636.

Cristina Rottondi, Leonardo Severi, Massimiliano Zanoni

Version: 2026
©2025-26

README

Lab 3 Exercise 1: Gesture-Based Regression

Gesture-Based Regression Using Processing, OSC Communication,
Wekinator, and Real-Time Circle Visualization

Final_Report.md

Contents

1	Title	2
2	Abstract	2
3	Exercise Objective	2
4	System Overview	2
5	Software and Libraries Used	3
6	Processing Input Sketch	3
7	6.1 Purpose of the Input Sketch	3
8	6.2 Input Features	4
9	Feature 1 — Normalized X Position	4
10	Feature 2 — Normalized Y Position	4
11	Feature 3 — Normalized Movement Speed	4
12	6.3 Input Normalization	4
13	6.4 OSC Output from the Input Sketch	5
14	Wekinator Project	5
15	7.1 Purpose of Wekinator	5
16	7.2 Wekinator Input Configuration	5
17	7.3 Wekinator Output Configuration	6
18	7.4 Model Type	6
19	Processing Output Sketch	6
20	8.1 Purpose of the Output Sketch	6
21	8.2 OSC Input to the Output Sketch	7
22	8.3 Mapping Output 1 to Circle Size	7
23	8.4 Mapping Output 2 to Outline Alpha	7
24	8.5 Visual Result	7
25	Training Methodology	8
26	9.1 Training Concept	8
27	9.2 Correct Training Plan	8
28	9.3 Practical Training Steps	9

29 Mistake Found During Development	9
30 Fix Applied to the Training Process	9
31 Results	10
32 Analysis and Discussion	10
33 13.1 Regression Behavior	10
34 13.2 Importance of Good Training Examples	10
35 13.3 Effect of Position Features	11
36 13.4 Effect of Speed Feature	11
37 13.5 OSC Communication	11
38 13.6 Limitations	11
39 Testing and Verification	12
40 Test 1 — Input Sketch Values	12
41 Test 2 — Wekinator Input Reception	12
42 Test 3 — Wekinator Training	12
43 Test 4 — Output Sketch Reception	12
44 Test 5 — Visual Result	13
45 Troubleshooting Notes	13
46 Problem: Output Values Stay at 0.500	13
47 Problem: Circle Does Not Change After Training	13
48 Problem: Wekinator Does Not Receive Input	13
49 Final Deliverables	13
50 Conclusion	14

1. Title

Gesture-Based Regression Using Processing, OSC Communication, Wekinator, and Real-Time Circle Visualization

2. Abstract

This report describes the implementation of **Lab 3 — Exercise 1: Gesture-Based Regression with Wekinator**. The goal of the exercise was to build an interactive machine learning system where mouse gestures are converted into continuous visual changes. The system uses two Processing sketches and one Wekinator project. The first Processing sketch captures mouse movement data, extracts three normalized gesture features, and sends them to Wekinator through OSC. Wekinator learns a regression mapping from these gesture features to two continuous output values. The second Processing sketch receives the predicted outputs from Wekinator and uses them to control the size and outline transparency of a circle.

The final system demonstrates the difference between classification and regression. Instead of assigning a gesture to a fixed category, the gesture is interpreted as a continuous input signal. This allows the visual output to change smoothly according to the mouse position and movement speed.

3. Exercise Objective

The objective of this exercise was to explore **regression in machine learning** through an interactive visual system.

The exercise required a system where mouse gestures control visual parameters continuously. The required mapping was:

```
Mouse gesture features → Visual circle properties
```

The gesture was represented by three input features:

1. Horizontal mouse position, x
2. Vertical mouse position, y
3. Mouse movement speed

The visual output was represented by two continuous values:

1. Circle **size**
2. Circle outline transparency / alpha

The complete system was designed so that Wekinator learns the relationship between mouse movement and visual output through supervised training examples.

4. System Overview

The system contains three main parts:

1. Processing input sketch
2. Wekinator regression project
3. Processing output sketch

The full data flow is:

```
Mouse movement
  ↓
Processing input sketch
  ↓
```

```

Feature extraction: x, y, speed
↓
Normalization between 0.0 and 1.0
↓
OSC message sent to Wekinator
↓
Wekinator regression model
↓
Two continuous output predictions
↓
OSC message sent to output sketch
↓
Circle size and outline alpha change in real time

```

This structure separates the project into three clear responsibilities:

Component	Responsibility
Processing input sketch	Captures and sends gesture features
Wekinator	Learns and predicts the regression mapping
Processing output sketch	Visualizes the predicted outputs

5. Software and Libraries Used

The implementation used the following tools:

Tool / Library	Purpose
Processing	Creative coding environment used for input and output sketches
oscP5	Processing library used for OSC communication
netP5	Networking support library used with oscP5
Wekinator	Interactive machine learning software used for supervised regression
OSC	Communication protocol used between Processing and Wekinator

The Processing sketches communicate with Wekinator using **OSC**, which stands for **Open Sound Control**. OSC is useful in interactive multimedia systems because it allows real-time transmission of numerical data between programs.

6. Processing Input Sketch

7. 6.1 Purpose of the Input Sketch

The input Processing sketch is responsible for capturing the mouse gesture and converting it into numerical values that Wekinator can use.

The sketch extracts three features:

```

input-1 → normalized x position
input-2 → normalized y position
input-3 → normalized movement speed

```

These values are sent continuously to Wekinator.

8. 6.2 Input Features

9. Feature 1 — Normalized X Position

The horizontal mouse position is calculated from Processing's built-in `mouseX` value.

```
xNorm = mouseX / (float) width;
```

This converts the x position from pixels into a normalized value between 0.0 and 1.0.

Example:

```
mouseX = 0           → xNorm = 0.0  
mouseX = width / 2    → xNorm = 0.5  
mouseX = width        → xNorm = 1.0
```

This feature tells Wekinator where the mouse is horizontally inside the input window.

10. Feature 2 — Normalized Y Position

The vertical mouse position is calculated from Processing's built-in `mouseY` value.

```
yNorm = mouseY / (float) height;
```

This converts the y position from pixels into a normalized value between 0.0 and 1.0.

Example:

```
mouseY = 0           → yNorm = 0.0  
mouseY = height / 2   → yNorm = 0.5  
mouseY = height        → yNorm = 1.0
```

This feature tells Wekinator where the mouse is vertically inside the input window.

11. Feature 3 — Normalized Movement Speed

The movement speed is calculated using the distance between the current mouse position and the previous mouse position:

```
float speed = dist(mouseX, mouseY, pmouseX, pmouseY);
```

This measures how far the mouse moved between two consecutive frames. The value is then normalized:

```
speedNorm = speed / maxSpeed;
```

The value is constrained between 0.0 and 1.0:

```
speedNorm = constrain(speedNorm, 0, 1);
```

This feature allows Wekinator to distinguish between slow and fast gestures.

12. 6.3 Input Normalization

Normalization is important because the raw values have different scales:

```
mouseX: 0 to 640  
mouseY: 0 to 480  
speed: variable depending on movement
```

After normalization, all features are in the same range:

```
xNorm:      0.0 to 1.0
yNorm:      0.0 to 1.0
speedNorm:  0.0 to 1.0
```

This makes the input data more balanced and suitable for regression.

13. 6.4 OSC Output from the Input Sketch

The input sketch sends the three features to Wekinator using this OSC structure:

```
OSC address: /wek/inputs
Destination host: 127.0.0.1
Destination port: 6448
Number of values sent: 3
```

The values are sent in this order:

```
1. xNorm
2. yNorm
3. speedNorm
```

The order is important because Wekinator receives them as:

```
input-1 = xNorm
input-2 = yNorm
input-3 = speedNorm
```

14. Wekinator Project

15. 7.1 Purpose of Wekinator

Wekinator is the machine learning component of the system. It receives the three gesture input features from the Processing input sketch and learns how to predict two continuous output values.

The learned mapping is:

```
x position + y position + speed
      ↓
circle size + outline transparency
```

Wekinator is trained using supervised examples. During training, the user moves the mouse to create input values and manually sets the desired output values.

16. 7.2 Wekinator Input Configuration

The correct input configuration is:

```
Input port: 6448
Input OSC message: /wek/inputs
Number of inputs: 3
```

The three inputs are:

```
input-1 → normalized x position
input-2 → normalized y position
input-3 → normalized movement speed
```

The OSC In indicator in Wekinator should show that it is receiving data when the Processing input

sketch is running.

17. 7.3 Wekinator Output Configuration

The correct output configuration is:

```
Output OSC message: /wek/outputs
Output host: localhost
Output port: 12000
Number of outputs: 2
Output type: All continuous
```

The two outputs are:

```
outputs-1 → circle size
outputs-2 → outline alpha / transparency
```

The output type must be continuous because the goal is to produce smooth numerical values, not discrete class labels.

18. 7.4 Model Type

The lab specification requires a regression model for mapping gesture features to continuous outputs. The intended model is a **K-Nearest Neighbors regressor with $k = 3$** .

During implementation, the installed Wekinator version showed only the following options for continuous numeric output models:

```
Neural Network
Linear/Polynomial Regression
```

The KNN regression option was not available in the visible model-selection dropdown for this continuous-output project. Because the project still required continuous regression behavior, the implementation used the available **Neural Network regression model** while keeping the required number of inputs, number of outputs, OSC messages, ports, and continuous output type.

If a Wekinator version provides KNN regression for continuous outputs, the correct setting according to the lab specification should be:

```
Model type: KNN Regression
k = 3
```

The important point in the implemented system is that the project remained a regression system, not a classification system.

19. Processing Output Sketch

20. 8.1 Purpose of the Output Sketch

The output Processing sketch receives Wekinator's two regression outputs and visualizes them as a circle.

The sketch receives:

```
outputs-1 → circle size
outputs-2 → outline alpha / transparency
```

It does not capture input data and does not train the model. Its only purpose is visualization.

21. 8.2 OSC Input to the Output Sketch

The output sketch listens for OSC messages from Wekinator using:

```
OSC address: /wek/outputs  
Listening port: 12000
```

When a message is received, the sketch reads two values:

```
circleSizeOutput = theOscMessage.get(0).floatValue();  
outlineAlphaOutput = theOscMessage.get(1).floatValue();
```

These values are expected to be between 0.0 and 1.0.

22. 8.3 Mapping Output 1 to Circle Size

The first Wekinator output controls the circle diameter.

```
circleSize = map(circleSizeOutput, 0, 1, 30, 350);
```

This means:

```
circleSizeOutput = 0.0 → circle size = 30 pixels  
circleSizeOutput = 0.5 → circle size = 190 pixels  
circleSizeOutput = 1.0 → circle size = 350 pixels
```

The minimum size is set to 30 pixels so the circle does not disappear completely.

23. 8.4 Mapping Output 2 to Outline Alpha

The second Wekinator output controls the circle outline transparency.

```
outlineAlpha = map(outlineAlphaOutput, 0, 1, 0, 255);
```

This means:

```
outlineAlphaOutput = 0.0 → alpha = 0, invisible outline  
outlineAlphaOutput = 0.5 → alpha = 127.5, semi-transparent outline  
outlineAlphaOutput = 1.0 → alpha = 255, fully visible outline
```

This directly satisfies the exercise requirement that the outline transparency should be controlled by Wekinator.

24. 8.5 Visual Result

The output sketch draws a circle in the center of the window:

```
ellipse(width / 2, height / 2, circleSize, circleSize);
```

The circle has no fill:

```
noFill();
```

The outline is white with variable transparency:

```
stroke(255, outlineAlpha);
```

As a result:

```
Higher output 1 → larger circle
```

```
Lower output 1 → smaller circle
Higher output 2 → more visible outline
Lower output 2 → more transparent outline
```

25. Training Methodology

26. 9.1 Training Concept

Training Wekinator means giving it examples of the desired relationship between input gestures and output values.

The user does not manually type the input values. The inputs come automatically from the Processing input sketch.

The user manually sets the output target values in Wekinator before recording.

During recording, Wekinator stores pairs like this:

```
current x, y, speed → chosen circle size, chosen outline alpha
```

After enough examples are recorded, Wekinator can learn the mapping and predict outputs for new mouse gestures.

27. 9.2 Correct Training Plan

The corrected training plan used different mouse regions and different output values.

A good training set is:

```
Recording 1:
Mouse area: top-left
outputs-1 = 0.2
outputs-2 = 0.2
Visual meaning: small circle, transparent outline

Recording 2:
Mouse area: top-right
outputs-1 = 0.8
outputs-2 = 0.2
Visual meaning: large circle, transparent outline

Recording 3:
Mouse area: bottom-left
outputs-1 = 0.2
outputs-2 = 0.8
Visual meaning: small circle, strong outline

Recording 4:
Mouse area: bottom-right
outputs-1 = 0.8
outputs-2 = 0.8
Visual meaning: large circle, strong outline

Optional Recording 5:
Mouse area: center
outputs-1 = 0.5
outputs-2 = 0.5
Visual meaning: medium circle, medium outline
```

This training plan is useful because Wekinator receives different examples for different regions of the input space.

28. 9.3 Practical Training Steps

The final training workflow was:

1. Run the Processing input sketch.
2. Run the Processing output sketch.
3. Open the Wekinator project.
4. Check that Wekinator receives OSC input.
5. Set outputs-1 and outputs-2 to the desired values.
6. Click Start Recording.
7. Move the mouse inside the input Processing window.
8. Stop recording after 3-5 seconds.
9. Change the output values.
10. Repeat with a different mouse region or movement style.
11. Click Train.
12. Click Run.
13. Move the mouse and observe the circle output.

Only the input Processing window needs mouse movement during testing. The output Processing sketch only receives predictions and draws the circle.

29. Mistake Found During Development

A major issue was discovered during the first training attempt.

The Wekinator project settings were mostly correct, but all recorded examples had the same output target values:

```
outputs-1 = 0.2  
outputs-2 = 0.2
```

Even though the mouse was moved in different directions, the target outputs were not changed between recordings.

Therefore, the model learned:

```
Every gesture → 0.2, 0.2
```

As a result, after training and running the model, the output values stayed almost fixed and the circle did not change.

This was not caused by the Processing input sketch.

This was not caused by the Processing output sketch.

It was a training-data problem.

30. Fix Applied to the Training Process

The training data was corrected by assigning different output targets to different mouse gestures.

Instead of recording:

```
Gesture 1 → 0.2, 0.2  
Gesture 2 → 0.2, 0.2  
Gesture 3 → 0.2, 0.2  
Gesture 4 → 0.2, 0.2
```

we used:

```
Gesture 1 → 0.2, 0.2  
Gesture 2 → 0.8, 0.2  
Gesture 3 → 0.2, 0.8
```

Gesture 4 → 0.8, 0.8

This gave Wekinator meaningful supervised examples.

After correction, the model had different desired outputs for different regions of the mouse gesture space, so it could learn a non-constant mapping.

31. Results

After correcting the training process, Wekinator was able to produce changing output values when the mouse moved in the input sketch.

The expected behavior was observed:

Moving the mouse to different regions changed the Wekinator output values.
 Output 1 controlled the circle **size**.
 Output 2 controlled the outline transparency.
 The circle changed continuously instead of jumping between fixed classes.

The output Processing sketch displayed the raw and mapped values, which helped verify the behavior.

For example:

Output 1 - circle **size** raw = changing value between 0.0 and 1.0
 Output 2 - outline alpha raw = changing value between 0.0 and 1.0
 Mapped circle **size** = value between 30 and 350 pixels
 Mapped outline alpha = value between 0 and 255

The final system successfully demonstrated a real-time gesture-based regression pipeline.

32. Analysis and Discussion

33. 13.1 Regression Behavior

The system demonstrates regression because the output values are continuous.

The model does not select one fixed label. Instead, it predicts numerical values that can vary smoothly. This is appropriate for visual control because circle size and outline transparency are naturally continuous parameters.

For example, the circle does not need to be only:

small
 medium
 large

It can have many intermediate sizes.

Similarly, the outline transparency can range smoothly from invisible to fully visible.

34. 13.2 Importance of Good Training Examples

The quality of the regression result depends strongly on the training examples.

The first failed training attempt showed that if all examples have the same output value, Wekinator learns a constant output.

This is an important machine learning lesson:

A model can only learn the relationships that are present in the training data.

If the training data says every gesture should produce the same result, then the trained model will produce the same result.

The corrected training data gave Wekinator a clearer relationship between mouse region and desired output.

35. 13.3 Effect of Position Features

The x and y position features made it possible to connect different parts of the Processing window to different visual outputs.

For example:

```
top-left area    → small and transparent circle
top-right area   → large and transparent circle
bottom-left area → small and visible circle
bottom-right area → large and visible circle
```

This allowed the model to learn a spatial mapping.

36. 13.4 Effect of Speed Feature

The speed feature added information about the motion style of the gesture.

When the mouse was still, the speed value was close to:

```
0.0
```

When the mouse moved quickly, the speed value increased toward:

```
1.0
```

This feature allowed the model to consider not only where the mouse was, but also how fast it was moving.

However, during training, it was important to avoid recording too many still frames because too many examples with speed equal to zero could make the speed feature less useful.

37. 13.5 OSC Communication

The OSC communication structure worked as a bridge between Processing and Wekinator.

The input sketch sent data to Wekinator using:

```
/wek/inputs on port 6448
```

Wekinator sent data to the output sketch using:

```
/wek/outputs on port 12000
```

The system depended on these values matching correctly across all programs. If the port or OSC message address was wrong, the system would not communicate.

38. 13.6 Limitations

The system works well as a simple demonstration of gesture-based regression, but it has some limitations.

First, the mouse gesture is represented by only three features. These are simple and easy to understand, but they do not capture complex gesture shapes.

Second, the model behavior depends heavily on how training examples are recorded. Poorly distributed

examples can produce weak or confusing output behavior.

Third, the installed Wekinator version did not expose KNN regression for continuous outputs in the model-selection interface, so the implementation used the available continuous regression model instead.

Fourth, if the user records examples while the mouse is still, the speed feature may not contribute much to the model.

39. Testing and Verification

The final system was checked using the following tests.

40. Test 1 — Input Sketch Values

The input sketch was run and the mouse was moved inside the Processing window.

Expected behavior:

```
x normalized changes when moving left/right
y normalized changes when moving up/down
speed normalized increases when moving faster
```

This confirmed that the input features were being calculated correctly.

41. Test 2 — Wekinator Input Reception

Wekinator was checked while the input sketch was running.

Expected behavior:

```
OSC In becomes active
The three input values change when the mouse moves
```

This confirmed that Wekinator was receiving data from Processing.

42. Test 3 — Wekinator Training

Several recordings were made with different output target values.

Expected behavior:

```
The number of recorded examples increases
The model can be trained after examples are recorded
The Run button becomes available after training
```

This confirmed that Wekinator had training data.

43. Test 4 — Output Sketch Reception

After training and clicking Run, the output sketch was checked.

Expected behavior:

```
Output 1 raw value changes
Output 2 raw value changes
Mapped circle size changes
Mapped outline alpha changes
```

This confirmed that the output sketch was receiving predictions from Wekinator.

44. Test 5 — Visual Result

The mouse was moved in the input sketch window after the model was running.

Expected behavior:

```
The circle changes size.  
The circle outline becomes more or less transparent.  
The changes are continuous.
```

This confirmed that the complete system was working.

45. Troubleshooting Notes

46. Problem: Output Values Stay at 0.500

This usually means the output sketch has not received messages from Wekinator yet.

Possible fixes:

```
Make sure Wekinator is trained.  
Click Run in Wekinator.  
Make sure the output sketch is running.  
Make sure Wekinator sends to port 12000.  
Make sure the OSC output message is /wek/outputs.
```

47. Problem: Circle Does Not Change After Training

This can happen if all training examples used the same output targets.

Fix:

```
Delete or replace the recordings.  
Record new examples with different output values.  
Train again.  
Run again.
```

48. Problem: Wekinator Does Not Receive Input

Possible causes:

```
Input sketch is not running.  
Wekinator input port is not 6448.  
Input OSC message is not /wek/inputs.  
The input sketch is sending to the wrong port.
```

Fix:

```
Run the input sketch.  
Check Wekinator input port.  
Check the OSC message address.
```

49. Final Deliverables

The final deliverables for Exercise 1 are:

1. Processing input sketch
2. Processing output sketch
3. Wekinator project

4. Final report

The recommended folder structure is:

```
Lab3_Exercise1/  
  Exercise1_GestureInput/  
    Exercise1_GestureInput.pde  
  
  Exercise1_CircleOutput/  
    Exercise1_CircleOutput.pde  
  
  Wekinator_Project/  
    Wekinator project files  
  
  Final_Report.md
```

50. Conclusion

This exercise implemented a complete real-time gesture-based regression system using Processing, OSC, and Wekinator.

The input Processing sketch captured three mouse gesture features: normalized x position, normalized y position, and movement speed. These values were sent to Wekinator using OSC. Wekinator learned a regression mapping from the three gesture inputs to two continuous visual outputs. The output Processing sketch received these two predictions and used them to control the size and outline transparency of a circle.

The most important result of the exercise was understanding how supervised regression depends on meaningful training examples. The first training attempt produced a nearly constant output because all examples had the same target values. After correcting the training data by assigning different outputs to different mouse regions and movements, the system produced changing visual results.

The final system demonstrates the main idea of regression in an interactive multimedia context: continuous input gestures can be mapped to continuous visual parameters, creating smooth and expressive real-time control.